

## 1. Processing Objective and Analytical Question

Part 1 analysed the *Clothing, Shoes and Jewelry* category of the Amazon Reviews 2023 dataset [1, 2]. Part 2 extends this work by using PySpark and MapReduce concepts to analyse customer feedback at product level. The analysis uses the same 3 GiB processing subset, containing 7,231,543 reviews.

The analytical question is:

*Which products show the highest customer risk when considering review volume, negative reviews, helpful votes on negative reviews, and changes in ratings over time?*

The aim is to transform and aggregate the review data into product-level measures and use these measures to identify products that may require further investigation. Products with at least 50 reviews are considered for the final ranking. This threshold was selected because Part 1 found that the 99th percentile was 48 reviews per product, providing a practical way of focusing on products with sufficient review evidence. The threshold is therefore a filtering rule rather than a statistical significance test.

The intended outcome is a product-level ranking that combines several customer-feedback signals, allowing products with different types of warning signs to be compared using a common measure.

## 2. Mapping and Transformation

Each review was transformed into variables required for the product-level analysis. Ratings of one or two stars were classified as negative reviews, and helpful votes associated with these reviews were recorded separately. Numeric indicators were also created for positive reviews and verified purchases.

Reviews were divided into two periods: *historical* (2020 and earlier) and *recent* (2021–2023). For products represented in both periods, rating change was calculated as:

$$\text{Rating Change} = \text{Recent Mean Rating} - \text{Historical Mean Rating}. \quad (1)$$

Negative values therefore represent lower recent ratings, while positive values represent improvement.

These measures capture different aspects of customer feedback. Negative reviews indicate customer dissatisfaction, helpful votes show whether other customers found negative feedback useful, and rating change captures whether customer evaluations are becoming better or worse over time.

The MapReduce process was demonstrated explicitly using the Spark RDD API. Each review was mapped to a key-value pair with `parent_asin` as the key. The associated values contained the measures required for later aggregation, including rating, negative-review status, helpful votes on negative reviews, and verified-purchase status. These transformations convert individual review records into values directly relevant to the analytical question.

## 3. Shuffle, Grouping and Reduction

Reviews for the same product may initially be stored in different Spark partitions. They must therefore be redistributed and grouped by `parent_asin` before product-level measures can be calculated. This redistribution represents the shuffle stage. This follows the MapReduce principle of grouping intermediate values by key before reduction [3].

The explicit RDD implementation follows:

$$\text{Map} \rightarrow \text{Shuffle} \rightarrow \text{Reduce}. \quad (2)$$

The `map()` transformation creates product key-value pairs, while `reduceByKey()` brings matching keys together and combines their values. The reduction produces product-level review counts, rating sums, negative-review counts, helpful votes on negative reviews, and verified purchase counts.

Using `reduceByKey()` is appropriate because the required product-level measures can be combined incrementally across partitions, rather than collecting all reviews for a product into a single list. The resulting aggregates provide the measures required to calculate negative-review share, mean ratings, rating change, and the final customer-risk score.

The RDD reduction produced 1,836,721 parent products. Equivalent aggregations were performed using `DataFrame groupBy()` and `agg()` operations. Selected results from the two implementations were compared and produced zero mismatches for the checked measures, providing evidence that the aggregations were consistent.

Products with at least 50 reviews were then ranked using:

$$\begin{aligned} \text{Risk Score} = & \text{Negative Share} \times \log(1 + \text{Review Count}) \\ & \times [1 + \log(1 + \text{Negative Helpful Votes})] \\ & \times [1 + \max(-\text{Rating Change}, 0)]. \end{aligned} \quad (3)$$

The score increases with negative-review share, review evidence, helpful negative feedback, and recent rating deterioration. Log transformations reduce the influence of extremely large review and helpful-vote counts. Where rating change could not be calculated, no deterioration penalty was added.

The score is an interpretable prioritisation measure rather than a predictive model. A high score identifies a product that may require further investigation but does not establish that the product is defective.

#### 4. Spark Execution and DAG

Spark uses lazy evaluation, meaning that transformations build an execution plan without immediately processing the data. Execution begins when an action requires a result. Spark represents the required operations as a Directed Acyclic Graph (DAG), allowing distributed transformations to be organised into stages and tasks [4]. The execution process can be summarised as:

$$\text{Action} \rightarrow \text{Job} \rightarrow \text{Stages} \rightarrow \text{Tasks}. \quad (4)$$

A job may contain several stages, while each stage contains tasks that operate on data partitions. Narrow transformations such as `map()` can normally operate within existing partitions without redistributing records. In contrast, key-based aggregation is a wide operation because reviews with the same `parent_asin` may be located in different partitions.

Grouping these records therefore requires data to move between partitions, creating a shuffle. Such wide operations may introduce stage boundaries in Spark’s execution plan.

The formatted Spark physical plan was inspected using `explain(mode="formatted")`. Operations such as `Exchange` indicate redistribution of data between partitions, while `HashAggregate` represents aggregation. The Colab Spark runtime reported a default parallelism of 12, while 24 partitions were used for the main processing and shuffle operations. Traditional Hadoop MapReduce follows a Map–Shuffle–Reduce processing structure [3]. Spark can instead represent several connected transformations in a DAG and divide the resulting work into jobs, stages, and tasks [4]. This allows a sequence of transformations and aggregations to be managed as part of a larger execution plan. Supporting Spark execution evidence is provided in Appendix A.2.

#### 5. Results and Technical Interpretation

A total of 17,765 products met the 50-review threshold. Table 1 presents examples of the highest-ranked products.

Table 1: Examples of highly ranked products.

| Parent ASIN | Reviews | Negative (%) | Rating change | Risk score |
|-------------|---------|--------------|---------------|------------|
| B019VM3CPW  | 408     | 26.2         | -1.789        | 28.18      |
| B074TWL5HV  | 2,329   | 37.9         | -0.291        | 26.98      |
| B073D254G5  | 1,163   | 36.0         | -0.526        | 23.43      |

The highest-ranked product, B019VM3CPW, had a risk score of 28.18. Its average rating declined from approximately 3.79 historically to 2.00 recently, while 26.2% of its reviews were negative. B074TWL5HV had a smaller rating decline but a higher negative-review share of 37.9%, resulting in the second-highest score. The third-ranked product, B073D254G5, combined a 36.0% negative-review share with a decline of approximately 0.53 stars.

This illustrates the purpose of the composite score: a product does not need to have the highest value for every individual measure to receive a high ranking. Instead, the ranking reflects the combined strength of its warning signals.

Among the 16,535 eligible products for which both historical and recent ratings could be calculated, the mean rating change was approximately  $-0.118$  and the median was approximately  $-0.093$ . This indicates a small overall negative shift, although rating changes varied substantially between products.

The results answer the analytical question by identifying products that combine different warning signals. The highest-risk products are not simply those with the most reviews or the largest rating decline. Instead, different combinations of negative-review share, review evidence, helpful negative feedback, and rating deterioration contribute to the ranking.

#### 6. Visualisation

Figure 1 shows the 15 products with the highest risk scores. It provides a direct comparison of the products prioritised by the combined risk measure.

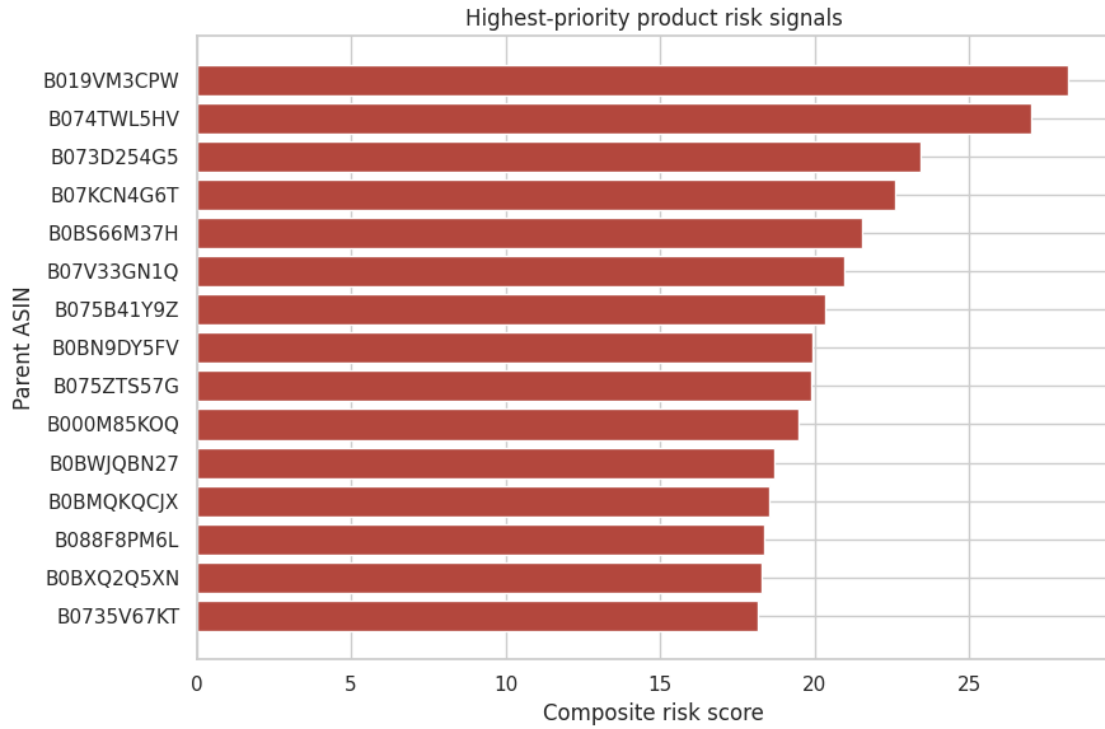


Figure 1: Top 15 products by customer-risk score.

This figure shows which products are prioritised by the composite score. It is a ranking for investigation, not evidence that any product is defective.

Figure 2 compares negative-review share with rating change for the 5,000 highest-volume eligible products, not a random sample of all eligible products. Values below zero on the vertical axis represent products whose recent average rating is lower than their historical average.

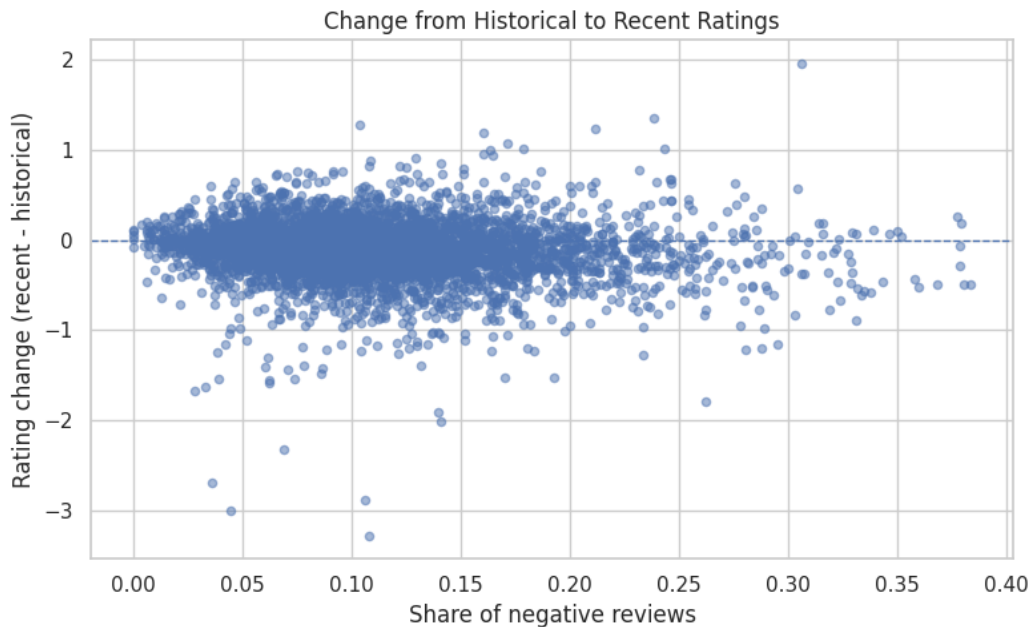


Figure 2: Negative-review share compared with rating change.

No strong simple relationship is visible between the two measures. Products with similar negative-review shares can show different rating changes, and products with declining ratings do not necessarily have the largest negative-review shares. This supports the use of multiple indicators rather than relying on a single measure.

The distribution of rating changes and the simplified comparison of declined, stable, and improved products are provided in Appendix A.1. These figures provide additional context for the product ranking without repeating the main results.

## 7. Business/Societal Interpretation and Limitations

The ranking can help retailers and manufacturers prioritise products for further investigation. Instead of manually examining large numbers of individual reviews, decision-makers can first examine products showing several warning

signals, such as a high negative-review share, substantial engagement with negative feedback, and declining ratings. Previous research has shown that online review characteristics such as ratings, volume, and helpfulness can provide useful information about product performance and consumer behaviour [5]. This may support product-quality monitoring, customer-experience analysis, and the prioritisation of further review.

For example, a retailer could use the ranking as a screening tool and examine the underlying complaints for the highest-ranked products before deciding whether supplier follow-up, additional quality checks, or closer monitoring is required.

Consumers may also benefit when aggregated review information is used to identify recurring concerns and improve product transparency. However, a high risk score should not be interpreted as proof that a product is defective. It indicates a combination of review signals that may justify closer investigation.

Several limitations should be considered. Reviews are voluntarily submitted and may be affected by self-selection, product or listing changes, repeated reviews, platform incentives, or manipulation. The processing dataset is a sequential subset rather than a random sample, so it may not fully represent the complete dataset.

## 8. Conclusion

The analysis shows that identifying products of concern requires more than looking at ratings alone. By combining negative-review share, review volume, helpful votes on negative reviews, and changes in ratings over time, the analysis highlighted products showing different patterns of customer risk. Of the 17,765 products that met the minimum review threshold, B019VM3CPW received the highest risk score of 28.18, with its average rating falling from approximately 3.79 historically to 2.00 recently. Across products with ratings available for both periods, the overall change was slightly negative, although the amount of change varied considerably between products.

PySpark made it possible to carry out this analysis across more than 7.2 million reviews while demonstrating how MapReduce concepts apply to large-scale data processing. Reviews were mapped by product, redistributed during the shuffle, and reduced into product-level measures. The agreement between the RDD and DataFrame results also provided a useful check that the aggregation had been implemented consistently.

Overall, the risk ranking provides a practical way of narrowing a large review dataset to products that deserve closer attention. It does not determine whether a product is defective, but it brings together several customer-feedback signals that would be difficult to assess individually at this scale. The results therefore show both the technical value of distributed processing and its practical use in turning large volumes of customer feedback into information that can support further investigation.

**Reproducibility:** The code, notebooks, figures, and supporting outputs are available in the [GitHub repository](#).

## References

- [1] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. Bridging language and items for retrieval and recommendation. *arXiv preprint arXiv:2403.03952*, 2024.
- [2] McAuley Lab. Amazon reviews 2023. Hugging Face, 2023. Accessed 2026.
- [3] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, pages 137–150, 2004.
- [4] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster computing with working sets. In *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing*, 2010.
- [5] K. Li, Y. Chen, and L. Zhang. Exploring the influence of online reviews and motivating factors on sales: A meta-analytic study and the moderating role of product category. *Journal of Retailing and Consumer Services*, 55:102107, 2020.

## A. Appendix

### A.1 Exploratory Visualisations

Figure 3 shows the distribution of changes between historical and recent ratings for the 5,000 high-volume products used in the visual analysis.

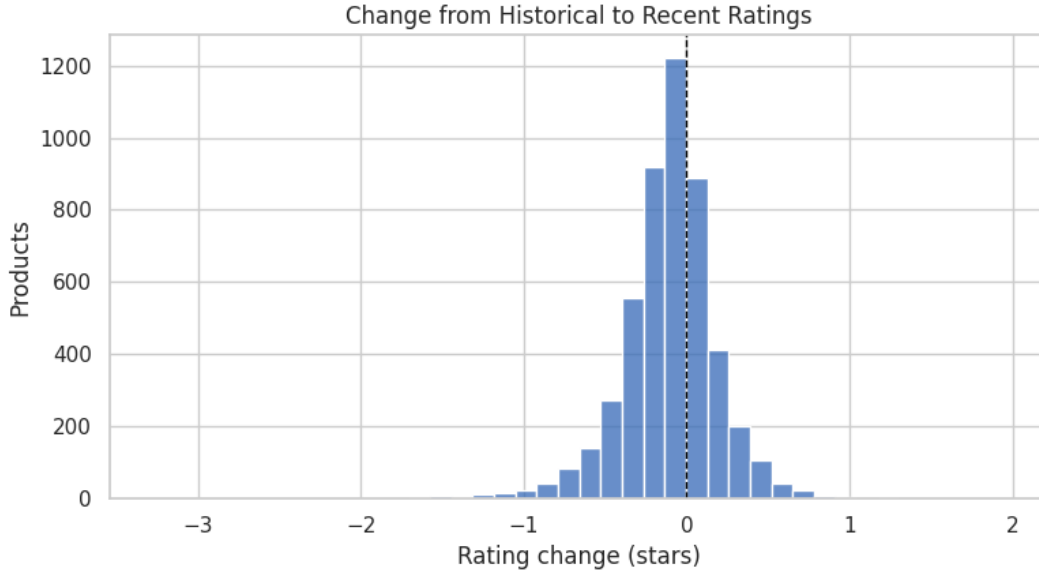


Figure 3: Change from historical to recent ratings for 5,000 high-volume products.

Negative values represent lower recent ratings, values close to zero represent relatively little change, and positive values represent higher recent ratings. The distribution shows that rating deterioration and improvement vary considerably across products.

Figure 4 provides a simpler summary of the same comparison.

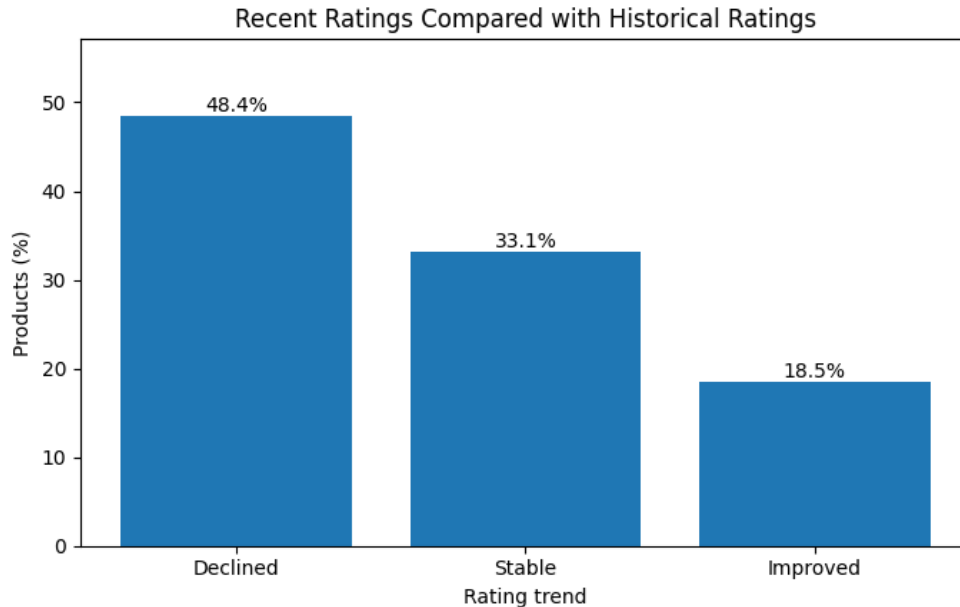


Figure 4: Recent ratings compared with historical ratings.

A product is classified as *Declined* when its rating change is below  $-0.1$ , *Stable* when the change is between  $-0.1$  and  $0.1$ , and *Improved* when the change is above  $0.1$ .

Of the 5,000 products analysed, 2,420 (48.4%) showed a decline in recent ratings, 1,653 (33.1%) remained relatively stable, and 927 (18.5%) showed an improvement. A change within  $\pm 0.1$  was treated as stable to allow for small differences between historical and recent ratings. This threshold is used only for descriptive comparison and does not represent statistical significance.

## A.2 Supporting Result Tables

The Spark physical plan was inspected using `explain(mode="formatted")`. The plan provides evidence of how Spark organised the transformations and aggregations used in the product-level analysis.

Operations such as `Exchange` indicate redistribution of records between partitions, while `HashAggregate` represents distributed aggregation. These operations are expected when reviews must be grouped by product.

The execution environment reported:

Table 2: Spark execution configuration.

| Measure                       | Result |
|-------------------------------|--------|
| PySpark version               | 4.0.0  |
| Default parallelism           | 12     |
| Configured shuffle partitions | 24     |
| Cached review partitions      | 24     |
| Executors reported            | 1      |

The absence of active jobs or stages after the actions completed does not mean that no jobs were executed. It indicates that the requested Spark actions had already finished when the execution status was inspected.

### A.2.1 Product-level Results

Table 3: Summary of product-level processing results.

| Measure                                   | Result    |
|-------------------------------------------|-----------|
| Parent products produced by RDD reduction | 1,836,721 |
| Products with at least 50 reviews         | 17,765    |
| Products with calculable rating change    | 16,535    |
| Mean rating change                        | -0.118    |
| Median rating change                      | -0.093    |
| Maximum risk score                        | 28.180    |
| RDD/DataFrame validation mismatches       | 0         |

The validation compares selected product-level measures calculated using the explicit RDD MapReduce implementation with equivalent DataFrame aggregations. The zero-mismatch result provides an additional correctness check for the implemented reduction.